

Homework #7 Solution

Spring of 2008

* This solution contains several problems in HW7.

References: From solution in the U of Texas A&M

1. 34.1-1

Suppose the LONGEST-PATH-LENGTH problem can be solved in polynomial time by an algorithm A . Given an instance of LONGEST-PATH $\langle G, u, v, k \rangle$, run the algorithm A on $\langle G, u, v \rangle$. Compare k with the result k' returned from A . $\langle G, u, v, k \rangle$ is a “YES” instance of LONGEST-PATH if and only if $k \leq k'$. The running time is the same as A , which is a polynomial.

Conversely, if the LONGEST-PATH problem can be solved in polynomial time by an algorithm A , then given an instance of LONGEST-PATH-LENGTH $\langle G, u, v \rangle$, run the algorithm A on $\langle G, u, v, k \rangle$ for $k = 1, 2, \dots, n - 1$. The smallest k such that $\langle G, u, v, k \rangle$ is a “YES” instance of LONGEST-PATH is the solution for the LONGEST-PATH-LENGTH problem on instance $\langle G, u, v \rangle$. It is clear that the running time is n times the running time of A , which is still a polynomial.

34.1-4

The dynamic programming algorithm of running time $O(nW)$ is not a polynomial time algorithm, because W is the maximum weight of items that the thief can put into his knapsack. Without loss of generality, assume that the input of the problem, W , is given as a binary number. Then the size of the input is $O(n \log W)$. The running time $O(nW)$ is not a polynomial in $O(n \log W)$ because for any fixed constant c , $nW > (n \log W)^c$ for sufficiently large W .

Comments on common errors: The W is not independent of the input size. It is given as part of the input. It is reasonable to assume that W is given as a binary number in the input. $\square$

34.1-6

If two languages $L_1, L_2 \in P$, then there exists algorithms A_1, A_2 that decide membership in L_1, L_2 , respectively, in polynomial time. Given an instance $I \in L_1 \cup L_2$, run the algorithm A_1 on I and then run A_2 on I . We decide $I \in L_1 \cup L_2$ if and only if A_1 or A_2 or both return “YES”.

If a language $L \in P$, then there exists an algorithm A that decides membership in L in polynomial time. Given an instance $I \in \bar{L}$, run the algorithm A on I . We decide $I \in L$ if and only if A returns “NO”. $\square$

2. 34.2-7 From Web

Show that the Hamiltonian-Path problem can be solved in polynomial-time on Directed-Acyclic-Graphs. Give an efficient algorithm for the problem: that is, your algorithm either gives a Hamiltonian path or it reports that there is no Hamiltonian path. Prove the correctness of your algorithm and analyze its running time.

Recall that a DAG must have at least one vertex with indegree 0. Note that a vertex with indegree 0 must be at the start of every path and, in particular, at the start of a Hamiltonian path.

So, if a DAG has two vertices u, v with indegree 0 then it can not have a Hamiltonian path (since they would both have to be at the start of the Hamiltonian path).

Now suppose that a DAG G does have a Hamiltonian Path $s = v_1, v_2, \dots, v_n$. Then node s must be the unique node with indegree 0 in G so it can easily be identified.

Furthermore, $v_2, \dots, v_n$ must be a Hamiltonian path in the DAG G' that results after deleting from G vertex s and all of the edges leaving it. From above we see that if G' has a Hamiltonian path then it must contain a unique vertex s' with indegree 0 and s' is the start of the Hamiltonian path.

This means that DAG G has a Hamiltonian Path $s = v_1, v_2, \dots, v_n$ if and only if

- a) s is the unique node with indegree 0 in G .*
- b) v_2 is the unique node with indegree 0 in G' .*
- c) $v_2, \dots, v_n$ is a Hamiltonian path in G' .*

This leads to the following algorithm

Create an array $in[v]$ which stores the in-degree of vertex v

This can be done in $O(n + e)$ time.

Search for vertex with $in[v] = 0$ and assign v to $ZeroIn$

This can be done in $O(n)$ time.

If more than one vertex with $in[v] = 0$, return 'No Hamiltonian Path'

This can be done in $O(n)$ time.

```

Set  $HP[] = \text{empty array}$ 
for  $i = 1$  to  $|V|$ 
     $HP[i] = \text{ZeroIn}$ 
     $\text{NumZeroIn} = 0$ 
     $\forall (ZeroIn, v) \in E$ 
         $in[v] = in[v] - 1$ 
        if  $in[v] == 0$ 
             $\text{NumZeroIn} = \text{NumZeroIn} + 1$ 
         $tmp = v$ 
    if  $\text{NumZeroIn} \neq 1$ 
        return 'No Hamiltonian Path'
    else  $\text{ZeroIn} = tmp$ 
return  $HP$ 

```

Running Time:

The running time is $O(n + e)$.